Dynamic Programming :-

It is a method that can be used when the solution to the problem can be viewed as a result of sequence of decision.

In greedy method, one decision sequence is generated where as in dynamic programming many decision sequence are generated.

In DP, the principle of optimately states that an optimal sequence of decision has the property that whatever the initial state & decision are, the remaining decision are must constitute an optimal decision sequence with regard to the state resulting from the first decision.

The problems (applications) of DP are :-

→ Multi-stage graph
→ Optimal Binary Search Tree (OBST)
→ 0/1 Knapsack Problem
→ All pairs - shortest path
→ Travelling Sales Person.
→ Reliability Design

All Pairs Shortest Path :-

Let $G = \{V, E\}$ be a directed graph with 'n' vertices, Let c be a cost of adjacent matrix for 'G' such that $cost(i,i) = 0$ where $1 \le i \le n$. $Cost(i,j)$ is the length of the edge (i,j) if $(i,j) \in \underset{\text{Edge}}{E}(G)$. $Cost(i,j) = \infty$ if $i \neq j$ & $(i,j) \notin E(G)$. The all pairs shortest path problem is to determine a matrix 'A' such that $A(i,j)$ is the length of shortest path from i to j The matrix 'A' can be obtained by solving 'n' single source problems using the algorithm shortest path.

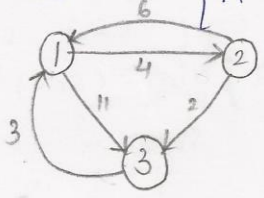
Since each application of this procedure requires $O(n^2)$ time, the matrix 'A' can be obtained in $O(n^3)$

Formulae :-

$$A^0(i,j) = cost(i,j) \quad 1 \le i \le n, \; 1 \le j \le n.$$

$$A^k(i,j) = \min\{A^{k-1}(i,j), A^{k-1}(i,k) + A^{k-1}(k,j)\} \quad k \ge 1$$

1>



Step 1 :-

$A^0(i,j) = cost(i,j)$

$A^0(1,1) = 0$ $\quad A^0(2,1) = 6$ $\quad A^0(3,1) = 3$

$A^0(1,2) = 4$ $\quad A^0(2,2) = 0$ $\quad A^0(3,2) = \infty$

$A^0(1,3) = 11$ $\quad A^0(2,3) = 2$ $\quad A^0(3,3) = 0$

Matrix :-

A^0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

Step 2 :-

$k=1$, $i=1$, $j=1,2,3$

$A^k(i,j) = \min\{A^k(i,j), A^{k-1}(i,k) + A^{k-1}(k,j)\}$

$A^1(1,1) = \min\{A^{1-1}(1,1), A^{1-1}(1,1) + A^{1-1}(1,1)\}$

$= \min\{A^0(1,1), A^0(1,1) + A^0(1,1)\}$

$= \min\{0\}$

$A^1(1,1) = 0$

A^0	1	2	3
1	0		
2			
3			

$\Rightarrow k=1$ $\quad i=1$, $\quad j=2$

$A^1(1,2) = \min\{A^{1-1}(1,2), A^{1-1}(1,1) + A^{1-1}(1,2)\}$

$= \min\{A^0(1,2), A^0(1,1) + A^0(1,2)\}$

$= \min\{4, 0+4\}$

$= \min\{4,4\}$

$A^1(1,2) = 4$

A^0	1	2	3
1	0	4	
2			
3			

$\Rightarrow k=1$ $\quad i=1$ $\quad j=3$

$A^1(1,3) = \min\{A^{1-1}(1,3), A^{1-1}(1,1) + A^0(1,3)\}$

$= \min\{A^0(1,3), A^0(1,1) + A^0(1,3)\}$

$= \min\{11, 0+11\}$

$A^1(1,3) = 11$

A^0	1	2	3
1	0	4	11
2			
3			

$\Rightarrow k=2$, $\quad i=2$, $\quad j=1$

$A^2(2,1) = \min\{A^{1}(2,1), A^{1}(2,2) + A^{1}(1,1)\}$

$= \min\{A^0(2,1), A^0(2,1) + A^0(1,1)\}$

$= \min\{6, 6+0\}$

$A^1(2,1) = \{6\}$

⇒ $k=1,\ i=2\ j=2$

$A^1(2,2) = \min\{A^{1-1}(2,2), A^{1-1}(2,1) + A^{1-1}(1,2)\} = \min\{0, 6+4\} = 0$

⇒ $k=1,\ i=2,\ j=3$

$A^1(2,3) = \min\{A^0(2,3), A^0(2,1) + A^0(1,3)\} = \min\{2, 6+4\} = 2$

⇒ $k=1,\ i=3,\ j=1$

$A^1(3,1) = \min\{A^0(3,1), A^0(3,1) + A^0(1,1)\} = \min\{3, 3+0\} = 3$

⇒ $k=1\quad i=3\quad j=2$

$A^1(3,2) = \min\{A^0(3,2), A^0(3,1) + A^0(1,2)\} = \min\{0, 7\} = 7.$

⇒ $k=1\quad i=3\quad j=3$

$A^1(3,3) = \min\{A^0(3,3), A^0(3,1) + A^0(1,3)\} = \min\{0, 3+11\} = 0$

A^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

Step 3 :-

⇒ $k=2,\ i=1,\ j=1$

$A^2(1,1) = \min\{A^{2-1}(1,1), A^{2-1}(1,2) + A^{2-1}(2,1)\}$

⇒ $k=2\ i=1, j=2 = \min\{A^1(1,1), A^1(1,2) + A^1(2,1)\} = \min\{0, 4+6\} = 0$

$A^2(1,2) = \min\{A^1(1,2), A^1(1,2) + A^1(2,2)\} = \min\{4, 4+0\} = 4$

⇒ ~~A~~ $k=2\quad i=1\quad j=3$

$A^2(1,3) = \min\{A^1(1,3), A^1(1,2) + A^1(2,3)\} = \min\{11, 4+2\} = 6$

⇒ $k=2\quad i=2\quad j=1$

$A^2(2,1) = \min\{A^1(2,1), A^1(2,2) + A^1(2,1)\}$

$= \min\{6, 0+6\} = 6$

⇒ $k=2\quad i=2\quad j=2$

$A^2(2,2) = \min\{A^1(2,2), A^1(2,2) + A^1(2,2)\}$

$= \min\{0, 0\}$

$= 0$

⇒ $k=2\quad i=2\quad j=3$

$A^2(2,3) = \min\{A^1(2,3), A^1(2,2) + A^1(2,3)\} = \min\{2, 0+2\}$

$= 2$

⇒ $K=2, i=3, j=1$

$A^2(3,1) = \min\{A^1(3,1), A^1(3,2)+A^1(2,1)\}$

$= \min\{3, 7+6\} = \min\{3, 13\} = 3$

⇒ $K=2, i=3, j=2$

$A^2(3,2) = \min\{A^1(3,2), A^1(3,2)+A^1(2,2)\}$

$= \min\{7, 7+0\} = 7$

⇒ $K=2 \quad i=3, j=3$

$A^2(3,3) = \min\{A^1(3,3), A^1(3,2)+A^1(2,3)\} = \min\{0, 7+2\} = 0$

A^2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

⇒ $K=3, i=1, j=1$

$A^3(1,1) = \min\{A^2(1,1), A^1(1,3)+A^1(3,1)\}$

$= \min\{0, 6+3\} = 0$

⇒ $K=3, \quad i=1, j=2$

$A^3(1,2) = \min\{A^2(1,2), A^2(1,3)+A^2(3,2)\} = \min\{4, 6+7\} = 4$

⇒ $K=3 \quad i=1 \quad j=3$

$A^3(1,3) = \min\{A^2(1,3), A^2(1,3)+A^2(3,3)\} = \min\{6, 0+6\} = 6$

⇒ $K=3 \quad i=2 \quad j=1$

$A^3(2,1) = \min\{A^2(2,1), A^2(2,3)+A^2(3,1)\} = \min\{6, 2+3\} = 5$

⇒ $K=3 \quad i=2 \quad j=2$

$A^3(2,2) = \min\{A^2(2,2), A^2(2,3)+A^2(3,2)\} = \{0\}$

⇒ $K=3 \quad i=2 \quad j=3$

$A^3(2,3) = \min\{A^2(2,3), A^2(2,3)+A^2(3,3)\} = \min\{2, 2+0\} = 2$

⇒ $K=3, i=3, j=1$

$A^3(3,1) = \min\{A^2(3,1), A^2(3,3)+A^2(3,1)\} = \min\{3, 0+3\} = 3$

⇒ $K=3, i=3, j=2$

$A^3(3,2) = \min\{A^2(3,2), A^2(3,3)+A^2(3,2)\} = \min\{7, 0+7\} = 7$

⇒ $K=3 \quad i=3 \quad j=3$

$A^3(3,3) = \min\{A^2(3,3), A^2(3,3)+A^2(3,3)\} = 0$

A^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

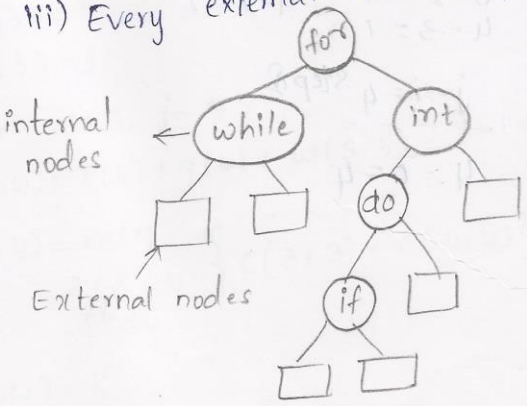
Algorithm Allpaths (cost, A, n)
{
    for i:1 to n do
    for j:1 to n do
    A[i,j]:=cost[i,j];
    for k:=1 to n do
    for i:=1 to n do
    for j:=1 to n do
        A[i,j]=min{A[i,j], A[i,k]+A[k,j]};

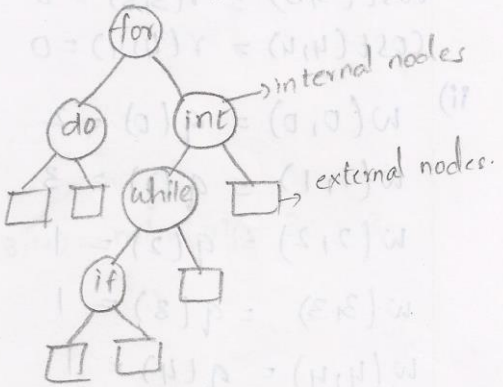
Optimal Binary Search Tree :

i) In a binary search tree for n identifiers there will be n internal nodes and n+1 external nodes.

ii) Every internal node represents a successful search

iii) Every external node represents a unsuccessful search.





In above diagram the probability of successful nodes or successful BST, varies from 1 to 2 diagrams. In order to overcome this we consider optimal binary search tree where we are going to find maximum cost for the respective routes.

Formulae :-

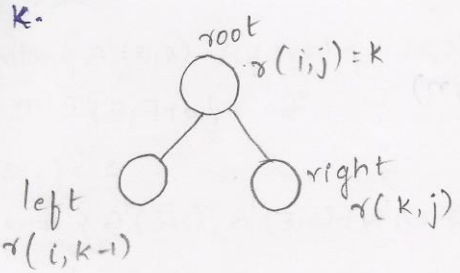
i) $cost(i,i) = r(i,i) = 0$

ii) $w(i,i) = q(i) \quad 0 \le i \le n$

3. $w(i,j) = p(j) + q(j) + w(i,j-1)$

4. $c(i,j) = \min_{i<k\le j} \{c(i,k-1) + c(k,j)\} + w(i,j)$

5. $r(i,j) = k$.



Problem :-

1. $n = 4$, $(a_1, a_2, a_3, a_4) = (do, if, int, while)$

$P(1:4) = 3, 3, 1, 1$

$q(0:4) = 2, 3, 1, 1, 1$ q values are $0 \le i \le n$ ($i = 0$ to 4)

$i = 0, 1, 2, 3, 4$

i) $cost(0,0) = r(0,0) = 0$
$cost(1,1) = r(1,1) = 0$
$cost(2,2) = r(2,2) = 0$
$cost(3,3) = r(3,3) = 0$
$cost(4,4) = r(4,4) = 0$

ii) $w(0,0) = q(0) = 2$
$w(1,1) = q(1) = 3$
$w(2,2) = q(2) = 1$
$w(3,3) = q(3) = 1$
$w(4,4) = q(4) = 1$

iii)

Step I	Step II	Step III
$j - i = 1$	$j - i = 2$	$j - i = 3$
$1 - 0 = 1$	$2 - 0 = 2$	$3 - 0 = 3$
$2 - 1 = 1$	$3 - 1 = 2$	$4 - 1 = 3$
$3 - 2 = 1$	$4 - 2 = 2$	
$4 - 3 = 1$		

$j - i = 4$ Step IV

$4 - 0 = 4$

⇒ For $j - i = 1$ (Step I)

3) $w(i,j) = p(j) + q(j) + w(i,j-1)$

* $i = 0$, $j = 1$ ($\because 1 - 0 = 1$)

$w(0,1) = p(1) + q(1) + w(0,0) = 3 + 3 + 2 = 8$ $\quad w(0,1) = 8$

4) $c(i,j) = \min_{i<k\le j} \{c(i,k-1) + c(k,j)\} + w(i,j)$

$$c(0,1) = \min_{0<k\le 1} \{c(0,0) + c(1,1)\} + w(0,1) = \min\{0+0\} + 8$$
$$= 8 + 0$$

$k = 1$

$$c(0,1) = 8$$

5) $r(i,j) = k$

$r(0,1) = 1$

* when $i = 1 \quad j = 2$

$w(1,2) = p(2) + q(2) + w(1,1)$

$= 3 + 1 + 3 = 7.$

$$c(1,2) = \min_{1<k\le 2} \{c(1,1) + c(2,2)\} + w(1,2) = \min\{0+0\} + w(1,2)$$

$k = 2 \qquad c(1,2) = 7.$

$r(1,2) = k$

$r(1,2) = 2.$

* when $i = 2 \quad j = 3$

$w(2,3) = p(3) + q(3) + w(2,2) = 1+1+1 = 3$

$$c(2,3) = \min_{2<k\le 3} \{c(2,2) + c(3,3)\} + w(2,3) = \min\{0+0\} + 3 = 3$$

$k = 3$

$r(2,3) = k$

$r(2,3) = 3$

* when $i = 3, \quad j = 4$

$w(3,4) = p(4) + q(4) + w(3,3) = 1+1+1 = 3.$

$$c(3,4) = \min_{3<k\le 4} \{c(3,3) + c(4,4)\} + w(3,4) = \min\{0\} + 3 = 3$$

$k = 4$

$r(3,4) = k$

$r(3,4) = 4$

Step II :- For $j - i = 2$

* $i = 0, \quad j = 2$

$w(0,2) = p(2) + q(2) + w(0,1)$

$= 3 + 1 + 8 = 12$

$$c(0,2) = \min_{0<k\le 2} \{c(0,0) + c(1,2)\} + w(0,2) = \min\{7+0, 8+0\}$$

$k = 1, k = 2 \qquad c(0,1) + c(2,2)$

$= \min\{7\} + 12$

$= 7 + 12 = 19$

$r(0,2) = 1$ ($\because k=1$ we got minimum value.

* when $i=1$ $j=3$

$w(1,3) = p(3) + q(3) + w(1,2)$

$= 1 + 1 + 7 = 9.$

$c(1,3) = \min_{1<k\le 3} \{c(1,1)+c(2,3), c(1,2)+c(3,3)\} + w(1,3)$

$k=2$, $k=3$

$= \min\{0+3, 7+0\} + 9$

$= 3 + 9 = 12$

$r(1,3) = k$

$r(1,3) = 2$

* when $i=2$ $j=4$

$w(2,4) = p(4) + q(4) + w(2,3)$

$= 1 + 1 + 3 = 5$

$c(2,4) = \min_{2<k\le 4} \{c(2,2)+c(3,4), c(2,3)+c(4,4)\} + w(2,4)$

$k=3$, $k=4$

$= \min\{0+3, 3+0\} + 5$

$= 3 + 5 = 8$

$r(2,4) = k$

$r(2,4) = 3$ (first 3 for $k=3$)

* when $i=$

Step III :-

* when $i=0$ $j=3$

$w(0,3) = p(3) + q(3) + w(0,2)$

$= 1 + 1 + 12 = 14$

$k=1$ $k=2$

$c(0,3) = \min_{0<k\le 3} \{c(0,0)+c(1,3), c(0,1)+c(2,3), c(0,2)+c(3,3)\} + w(0,3)$

$k=1,2,3$

$= \min\{0+12, 8+3, 19+0\} + 14 = \min\{12, 11, 19\} + 14 = 11 + 14$

12 11 19

$= 25$

$r(0,3) = k \qquad r(0,3) = 2$

* when $i = 1 \quad j = 4$

$w(1,4) = P(4) + q(4) + w(1,3)$

$= 1 + 1 + 9 = 11$

$c(1,4) = \min_{1 < k \le 4} \{ c(1,1) + c(2,4), c(1,2) + c(3,4), c(1,3) + c(4,4) \} + w(1,4)$ (k = 2, k = 3, k = 4)

$= \min\{ 8 + 0, 7 + 3, 12 + 3 \} + 11$

$= 8 + 11 = 19.$

$r(1,4) = 2$

Step IV :-

* when $i = 0 \quad j = 4$

$w(0,4) = P(4) + q(4) + w(0,3)$

$= 1 + 1 + 14 = 16$

$c(0,4) = \min_{0 < k \le 4} \{ c(0,0) + c(1,4), c(0,1) + c(2,4), c(0,2) + c(3,4), c(0,3) + c(4,4) \} + w(0,4)$

$k = 1$

$k = 2$

$k = 3$

$k = 4$

$= \min\{ 0 + 19, 8 + 8, 19 + 3, 25 + 0 \} + 16$

$= \min\{ 19, 16, 22, 25 \} = 16 + 11 = 32$

$r(0,4) = k$

$r(0,4) = 2$

	0	1	2	3	4
j−i = 0	w(0,0) = 2 c(0,0) = 0 r(0,0) = 0	w(1,1) = 3 c(1,1) = 0 r(1,1) = 0	w(2,2) = 1 c(2,2) = 0 r(2,2) = 0	w(3,3) = 1 r(3,3) = 0 r(3,3) = 0	w(4,4) = 1 c(4,4) = 0 r(4,4) = 0
j−i = 1	w(0,1) = 8 c(0,1) = 8 r(0,1) = 1	w(1,2) = 7 c(1,2) = 7 r(1,2) = 2	w(2,3) = 3 c(2,3) = 3 r(2,3) = 3	w(3,4) = 3 c(3,4) = 3 r(3,4) = 4	×
j−i = 2	w(0,2) = 12 c(0,2) = 19 r(0,2) = 1	w(1,3) 9 c(1,3) = 12 r(1,3) = 2	w(2,4) = 5 c(2,4) = 8 r(2,4) = 3	×	×
j−i = 3	w(0,3) = 14 c(0,3) = 25 r(0,3) = 2	w(1,4) = 11 c(1,4) = 19 r(1,4) = 2	×	×	×

Concept of dynamic programming:

Dynamic Programming(usually referred to as DP) is a powerful technique that allows to solve many different types of problems in time O(n2) or O(n3) for which a naive approach would take exponential time. In the word dynamic programming the word programming stands for "planning" and it does not mean by computer programming. Dynamic programming is typically applied to optimization problem.

Dynamic Programming is an algorithmic paradigm that solves a given complex problem by breaking it into subproblems and stores the results of subproblems to avoid computing the same results again. Following are the two main properties of a problem that suggest that the given problem can be solved using Dynamic programming.

1) Overlapping Subproblems
2) Optimal Substructure

1) Overlapping Subproblems:
Like Divide and Conquer, Dynamic Programming combines solutions to sub-problems. Dynamic Programming is mainly used when solutions of same subproblems are needed again and again. In dynamic programming, computed solutions to subproblems are stored in a table so that these don't have to recomputed. So Dynamic Programming is not useful when there are no common (overlapping) subproblems because there is no point storing the solutions if they are not needed again. For example, Binary Search doesn't have common subproblems. If we take example of following recursive program for Fibonacci Numbers, there are many subproblems which are solved again and again.

```
int fib(int n)
{
  if ( n <= 1 )
    return n;
  return fib(n-1) + fib(n-2);
}
```

2) Optimal Substructure
A problem is said to have optimal substructure if an optimal solution can be constructed efficiently from optimal solutions of its subproblems. This property is used to determine the usefulness of dynamic programming and greedy algorithms for a problem.

There are two ways of doing this.
1) Top-Down : Start solving the given problem by breaking it down. If you see that the problem has been solved already, then just return the saved answer. If it has not been solved, solve it and save the answer. This is usually easy to think of and very intuitive. This is referred to as Memoization.

Bottom-Up : Analyze the problem and see the order in which the sub-problems are solved and start

solving from the trivial subproblem , up towards the given problem. In this process, it is guaranteed that the subproblems are solved before solving the problem.

Dynamic Programming:
Note that divide and conquer is slightly a different technique. In that, we divide the problem in to non-overlapping subproblems and solve them independently, like in mergesort and quick sort.

Principal of optimality:

The principle of optimality states that no matter what the first decision, the remaining decisions must be optimal with respect to the state that results from this first decision.

This principle implies that an optimal decision sequence is comprised for some formulations of some problem.

Since the principle of optimality may not hold for some formulations of some problems, it is necessary to verify that it does not hold for the problem being solved.

Dynamic programming cannot be applied when this principle does not hold.

0/1 knapsack

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays val[0..n-1] and wt[0..n-1] which represent values and weights associated with n items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of val[] such that sum of the weights of this subset is smaller than or equal to W. You cannot break an item, either pick the complete item, or don't pick it (0-1 property).

Problem Description
If we are given n objects and a knapsack or a bag in which the object i that has weight w_i is to be placed. The knapsack has a capacity W. Then the profit that can be earned is $p_i x_i$. The objective is to obtain filling of knapsack with maximum profit earned.Maximized $p_i x_i$. subject to constraint $w_i x_i <= W$ Where $1<=i<=n$ and n is total no. of objects and $x_i = 0$ or 1.

Steps and Notations

Step-1:

Let $f_i(y_i)$ be the value of optimal solution. Then s^i is pair (p,w) where p=f(y_j) and w=y_j
Initially s^0 = {(0,0)}
We can compute s^{i+1} from s^i . These computations of s^i are basically the sequence of decisions made for obtaining the optimal solutions.

Step-2:
We can generate the sequence of decisions in order to obtain the optimum selection for solving the knapsack problem.

Let x_n be the optimum sequence. Then there are two instances $\{x_n\}$ and $\{x_{n-1},x_{n-2}\ldots x_1\}$. So from $\{x_{n-1},x_{n-2}\ldots x_1\}$ we will choose the optimum sequence with respect to x_n.
The selection of sequence from remaining set should be such that we should be able to fulfill the condition of filling knapsack capacity W with maximum profit.
Otherwise $\{x_{n-1},x_{n-2}\ldots x_1\}$ is not optimum.
This proves that 0/1 knapsack problem is solved using principle of optimality.

Step-3:
Let $f_i(y_j)$ be the value of optimal solution. Then fi(y) = max$\{f_{i-1}(y), f_{i-1}(y-w_i)+p_i\}$
Initially compute
$s^0=\{(0,0)\}$
s^i_1= {(P,W)|(P-p_1 , W-w_1) belongs to s^i}

S^{i+1} can be computed by merging s^i and s^i_1

Purging rule
If s^{i+1} contains (P_j, W_j) and (P_k, W_k) these two pairs such that Pj<=Pk and Wj>=Wk, then (P_j,W_j) can be eliminated . This purging rule is also called as dominance rule. In purging rule basically the dominated tuples gets purged. In short remove the pair with less profit and more weight.

Problem-1
Solve knapsack instance M=8, and n=4. let p_i and w_i are as shown below.

Solution-

$s^0=\{(0,0)\}$ initially
$s^0_1=\{(1,2)\}$
That means while building s^0_1 we select the next i^{th} pair. For s^0 we have selected first (P,W) pair which (1,2).

S^1 ={merge s^0 and s^0_1}
={(0,0),(1,2)}
s^1_1={select next (P,W) pair and add it with s^1}
={(2,3),(2+0,3+0), (2+1,3+2)}
s^1_1={(2,3),(3,5)} // repetition of (2,3) avoided.

S^2 ={merge candidates from s^1 and $s^1{}_1$}
={ (0,0), (1,2), (2,3), (3,5)}
$S^2{}_1$ ={select next (P,W) pair and add it with s^2}
={(5,4),(6,6),(7,7),(8,9)}
S^3 ={merge candidates from s^2 and $s^2{}_1$}
={ (0,0), (1,2),(2,3),(5,4),(6,6),(7,7),(8,9)}

Note that the pair (3, 5) is purged form s^3.
Because let us assume (P_j, W_j) = (3, 5) and (P_k, W_k) = (5, 4).
Here $P_j <= P_k$ and $W_j > W_k$ is true hence we will eliminate pair (P_j, W_j) = (3, 5) from s^3.

$S^3{}_1$ ={select next (P,W) pair and add it with s^3}
$S^3{}_1$ ={(6,5),(7,7),(8,8),(11,9),(12,11) ,(13,12) ,(14,14)}

S^4={(0,0),(1,2),(2,3) ,(5,4) ,(6,6) ,(7,7),(8,9) ,(6,5) ,(8,8),(11,9),(12,11) ,(13,12) ,(14,14)}
Now we are interested in M=8. We get pair (8, 8) in s^4. Hence
we will set x_4=1.

Now we select next object (P-p_4) and (W-w_4). i.e (8-6)
and (8-5).
i.e (2,3)
Pair (2,3) belongs to s^2. hence set x_2=1. So we get the final solution as (0, 1, 0, 1).

Travelling Salesman Dynamic Programming Algorithm

Let us consider a graph G = (V,E), where V is a set of cities and E is a set of weighted edges. An edge e(u, v) represents that vertices u and v are connected. Distance between vertex u and v is d(u, v), which should be non-negative.

Suppose we have started at city 1 and after visiting some cities now we are in city j. Hence, this is a partial tour. We certainly need to know j, since this will determine which cities are most convenient to visit next. We also need to know all the cities visited so far, so that we don't repeat any of them. Hence, this is an appropriate sub-problem.

For a subset of cities S ϵ{1,2,3,...,n} that includes 1, and j ϵ S, let C(S, j) be the length of the shortest path visiting each node in S exactly once, starting at 1 and ending at j.
When |S|> 1 , we define $\boldsymbol{C}$C(S,1)= ∝∝ since the path cannot start and end at 1.

Now, let express C(S, j) in terms of smaller sub-problems. We need to start at 1 and end at j. We should select the next city in such a way that

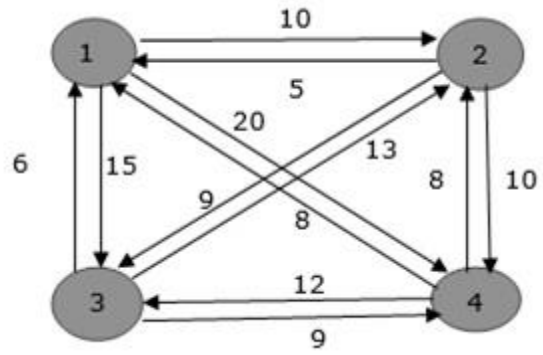
C(S,j)=minC(S−{j},i)+d(i,j)whereiϵSandi≠j(,)=(−{},)+(,)h≠

Algorithm: Traveling-Salesman-Problem
C ({1}, 1) = 0
for s = 2 to n do
  for all subsets S ϵ {1, 2, 3, … , n} of size s and containing 1
    C (S, 1) = ∞
  for all j ϵ S and j ≠ 1
    C (S, j) = min {C (S – {j}, i) + d(i, j) for i ϵ S and i ≠ j}
Return minj C ({1, 2, 3, …, n}, j) + d(j, i)
Analysis

There are at the most $2^n.n$ sub-problems and each one takes linear time to solve. Therefore, the total running time is $O(2^n.n^2)$.

Example

In the following example, we will illustrate the steps to solve the travelling salesman problem.



From the above graph, the following table is prepared.

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

S = ΦΦ

Cost(2,Φ,1)=d(2,1)=5����(2,Φ,1)=�(2,1)=5

Cost(3,Φ,1)=d(3,1)=6����(3,Φ,1)=�(3,1)=6

Cost(4,Φ,1)=d(4,1)=8����(4,Φ,1)=�(4,1)=8

S = 1

Cost(i,s)=min{Cos(j,s−(j))+d[i,j]}����(�,�)=���{���(�,�−(�))+�[�,�]}

Cost(2,{3},1)=d[2,3]+Cost(3,Φ,1)=9+6=15����(2,{3},1)=�[2,3]+����(3,Φ,1)=9+6=15

Cost(2,{4},1)=d[2,4]+Cost(4,Φ,1)=10+8=18����(2,{4},1)=�[2,4]+����(4,Φ,1)=10+8=18

Cost(3,{2},1)=d[3,2]+Cost(2,Φ,1)=13+5=18����(3,{2},1)=�[3,2]+����(2,Φ,1)=13+5=18

Cost(3,{4},1)=d[3,4]+Cost(4,Φ,1)=12+8=20����(3,{4},1)=�[3,4]+����(4,Φ,1)=12+8=20

Cost(4,{3},1)=d[4,3]+Cost(3,Φ,1)=9+6=15����(4,{3},1)=�[4,3]+����(3,Φ,1)=9+6=15

Cost(4,{2},1)=d[4,2]+Cost(2,Φ,1)=8+5=13����(4,{2},1)=�[4,2]+����(2,Φ,1)=8+5=13

S = 2

Cost(2,{3,4},1)=min{d[2,3]+Cost(3,{4},1)=9+20=29d[2,4]+Cost(4,{3},1)=10+15=25=25����(2,{3,4},1)=���{�[2,3]+����(3,{4},1)=9+20=29�[2,4]+����(4,{3},1)=10+15=25=25

Cost(3,{2,4},1)=min{d[3,2]+Cost(2,{4},1)=13+18=31d[3,4]+Cost(4,{2},1)=12+13=25=25����(3,{2,4},1)=���{�[3,2]+����(2,{4},1)=13+18=31�[3,4]+����(4,{2},1)=12+13=25=25

Cost(4,{2,3},1)=min{d[4,2]+Cost(2,{3},1)=8+15=23d[4,3]+Cost(3,{2},1)=9+18=27=23����(4,{2,3},1)=���{�[4,2]+����(2,{3},1)=8+15=23�[4,3]+����(3,{2},1)=9+18=27=23

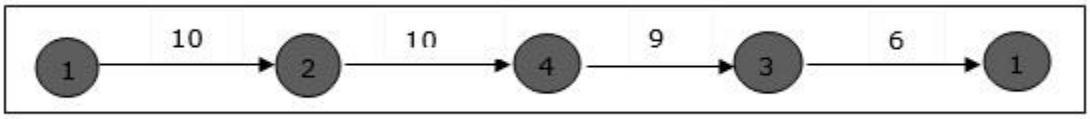
S = 3

Cost(1,{2,3,4},1)=min⎧⎩d[1,2]+Cost(2,{3,4},1)=10+25=35d[1,3]+Cost(3,{2,4},1)=15+25=40d[1,4]+Cost(4,{2,3},1)=20+23=43=35����(1,{2,3,4},1)=���{�[1,2]+����(2,{3,4},1)=10+25=35�[1,3]+����(3,{2,4},1)=15+25=40�[1,4]+����(4,{2,3},1)=20+23=43=35

The minimum cost path is 35.

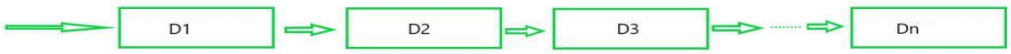
Start from cost {1, {2, 3, 4}, 1}, we get the minimum value for d [1, 2]. When s = 3, select the path from 1 to 2 (cost is 10) then go backwards. When s = 2, we get the minimum value for d [4, 2]. Select the path from 2 to 4 (cost is 10) then go backwards.

When s = 1, we get the minimum value for d [4, 2] but 2 and 4 is already selected. Therefore, we select d [4, 3] (two possible values are 15 for d [2, 3] and d [4, 3], but our last node of the path is 4). Select path 4 to 3 (cost is 9), then go to s = ϕ step. We get the minimum value for d [3, 1] (cost is 6).



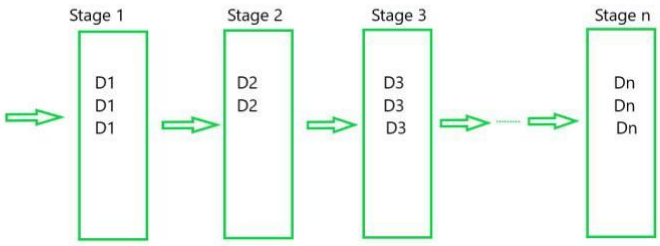
Reliability Design

The reliability design problem is the designing of a system composed of several devices connected in series or parallel. Reliability means the probability to get the success of the device.
Let's say, we have to set up a system consisting of D_1, D_2, D_3, ……, and D_n devices, each device has some costs C_1, C_2, C_3, ……., C_n. Each device has a reliability of 0.9 then the entire system has reliability which is equal to the product of the reliabilities of all devices i.e., $\pi r_i = (0.9)^4$.



Serial Combination of one copy of each device

It means that 35% of the system has a chance to fail, due to the failure of any one device. the problem is that we want to construct a system whose reliability is maximum. How it can be done so? we can think that we can take more than one copy of each device so that if one device fails we can use the copy of that device, which means we can connect the devices parallel.
When the same type of 3 devices is connected parallelly in stage 1 having a reliability 0.9 each then:
Reliability of device1, $r_1 = 0.9$
The probability that device does not work well = $1 - r_1 = 1 - 0.9 = 0.1$
The probability that all three copies failed = $(1-r_1)^3 = (0.1)^3 = 0.001$
The Probability that all three copies work properly = $1 - (1- r_1)^3 = 1- 0.001 = 0.999$
We can see that the system with multiple copies of the same device parallel may increase the reliability of the system.



Multiple copies of the same device type are connected in parallel through the use of a switching circuit

Given a cost C and we have to set up a system by buying the devices and we need to find number of copies of each device under the cost such that reliability of a system is maximized.

We have to design a three-stage system with device types D1, D2, and D3. The costs are $30, $15, and $20 respectively. The cost of the system is to be no more than $105. The reliability of each device type is 0.9, 0.8, and 0.5 respectively.

P_i	C_i	r_i
P_1	30	0.9
P_2	15	0.8
P_3	20	0.5

Explanation:

Given that we have total cost C = 105,

sum of all C_i = 30 + 15 + 20 = 65, the remaining amount we can use to buy a copy of each device in such a way that the reliability of the system, may increase.

Remaining amount = C – sum of C_i = 105 – 65 = 40
Now, let us calculate how many copies of each device we can buy with $40, If consume all $40 in device1 then we can buy 40/30 = 1 and 1 copy we have already so overall 2 copies of device1. Now In general, we can have the formula to calculate the upper bond of each device:

$U_i = \text{floor}(C - \sum C_i / C_i) + 1$ (1 is added because we have one copy of each device before)
C1=30, C2=15, C3=20, C=105
r1=0.9, r2=0.8, r3=0.5
u1 = floor ((105-(30+15+20))+1 = 2
u2 = 3
u3 = 3

A tuple is just an ordered pair containing reliability and total cost up to a choice of m_i's that has been made. we can make pair in of Reliability and Cost of each stage like ${}_{copy}S^{device}$
$S^0 = \{(1,0)\}$
Device 1:
Each S^i from S^{i-1} by trying out all possible values for r_i and combining the resulting tuples together.

- let us consider P_1 :
    - ${}_1S^1 = \{(0.9, 30)\}$ where 0.9 is the reliability of stage1 with a copy of one device and 30 is the cost of P_1.
- Now, two copies of device1 so, we can take one more copy as:
    - ${}_2S^1 = \{ (0.99, 60) \}$ where 0.99 is the reliability of stage one with two copies of device, we can see that it will come as: $1 - (1 - r_1)^2 = 1 - (1 - 0.9)^2 = 1 - 0.01 = 0.99$.
- After combining both conditions of Stage1 i.e., with copy one and copy of 2 devices respectively.

$S^1 = \{ (0.9, 30), (0.99, 60) \}$
Device 2:
S^2 will contain all reliability and cost pairs that we will get by taking all possible values for the stage2 in conjunction with possibilities calculated in S^1.
First of all we will check the reliability at stage2 when we have 1, 2, and 3 as a copy of device. let us assume that E_i is the reliability of the particular stage with n number of devices, so for S^2 we first calculate:

- E_2 (with copy 1) = $1 - (1 - r_2)^1 = 1 - (1 - 0.8) = 0.8$
- E_2 (with copy 2) = $1 - (1 - r_2)^2 = 1 - (1 - 0.8)^2 = 0.96$
- E_2 (with copy 3) = $1 - (1 - r_2)^3 = 1 - (1 - 0.8)^3 = 0.992$

If we use 1 copy of P_1 and 1 copy of P_2 reliability will be 0.9*0.8 and the cost will be 30+15
One Copy of Device two , ${}_1S^2 = \{ (0.8, 15) \}$ Conjunction with S^1 (0.9, 30) = { (0.72,45) }
Similarly, we can calculate other pairs as $S^2 = (0.72, 45), (0.792, 75), (0.864, 60), (0.98,90)$ }
We get ordered pair (0.98,90) in S^2 when we take 2 copies of Device1and 2 copies of Device2 However, with the remaining cost of 15 (105 – 90), we cannot use device Device3 (we need a minimum of 1 copy of every device in any stage), therefore (0.792, 75) should be discarded and other ordered pairs like it. We get $S^2 = \{ (0.72, 45), (0.864, 60), (0.98,90) \}$. There are other possible ordered pairs too, but all of them exceed cost limitations.
Up to this point we got ordered pairs:

S^1 = { (0.9, 30), (0.99, 60) }
S^2 = { (0.72, 45), (0.864, 60), (0.98,90)}

Device 3:

First of all we will check the reliability at stage3 when we have 1, 2, and 3 as a copy of device. E_i is the reliability of the particular stage with n number of devices, so for S^3 we first calculate:

- E_3 (with copy 1) = $1 - (1 - r_3)^1 = 1 - (1 - 0.5) = 0.5$
- E_3 (with copy 2) = $1 - (1 - r_3)^2 = 1 - (1 - 0.5)^2 = 0.75$
- E_3 (with copy 3) = $1 - (1 - r_3)^3 = 1 - (1 - 0.5)^3 = 0.875$

Now, possible ordered pairs of device three are as- S^3 = { (0.36, 65), (0.396, 95), (0.432, 80), (0.54, 85), (0.648, 100), (0.63, 105) }
(0.648,100) is the solution pair, 0.648 is the maximum reliability we can get under the cost constraint of 105.